SOEN 6011 Delivery 2 Report
F3: Hyperbolic Sine Function $\sinh(x)$

Arvind Lakshmanan
Student ID: 40310757
Concordia University

Summer 2026

Overview

This document gives the Delivery 2 answer for the assigned function F3, $\sinh(x)$. The D1 implementation used the direct exponential definition with a textual interface. For D2, the implementation is modified to satisfy the scratch implementation requirement, the Tkinter GUI requirement, exception handling, repository documentation, and updated requirements.

1 Problem 5: From-scratch Python implementation with Tkinter GUI

1.1 D2 implementation decision

The selected D2 algorithm is the Maclaurin series for the hyperbolic sine function:

$$\sinh(x) = x + \frac{x^3}{3!} + \frac{x^5}{5!} + \frac{x^7}{7!} + \cdots$$

This algorithm was selected for D2 because it can be implemented from scratch using only arithmetic operations. The implementation does not use `math.sinh`, `math.exp`, `math.factorial`, NumPy, SciPy, or other mathematical library functions. Tkinter is used for the graphical user interface only.

1.2 Why the D1 implementation was modified

The D1 implementation used the formula

$$\sinh(x) = \frac{e^x - e^{-x}}{2}.$$

That was suitable for D1 because D1 required a textual interface and did not yet require a scratch implementation. In D2, built-in or library mathematical functions are not used for calculating the

function. Therefore, the D2 implementation replaces the direct exponential method with a from-scratch Maclaurin method.

1.3 Subordinate functions implemented

The implementation includes subordinate functions that support the main function calculation:

- `absolute_value(value)`: returns a non-negative value without using `abs()`.
- `is_nan(value)`: detects NaN using the property that NaN is not equal to itself.
- `is_infinite_or_too_large(value)`: rejects infinity and unsafe huge values.
- `parse_supported_real_number(text)`: converts GUI input to a finite supported real number.
- `calculate_sinh_from_scratch(x_value)`: computes $\sinh(x)$ using a recurrence form of the Maclaurin series.

1.4 Series recurrence

Instead of recomputing powers and factorials from the beginning, the implementation uses the following recurrence:

$$\text{term}_0 = x$$

$$\text{term}_n = \text{term}_{n-1} \times \frac{x^2}{(2n)(2n+1)}$$

The running sum is updated by adding each term. The loop stops when the absolute value of the current term is smaller than the tolerance condition:

$$|\text{term}| \leq \text{TOLERANCE} \times (1 + |\text{sum}|).$$

1.5 Input range

The supported D2 range is:

$$-20 \leq x \leq 20.$$

This range is used because the scratch series implementation is accurate and responsive for ordinary calculator inputs in this interval. Larger ranges can be added later using more advanced numerical techniques, such as argument reduction.

1.6 Exception handling and helpful messages

The source code defines custom exception classes:

- `SinhInputError`: used for blank input, non-numeric input, NaN, infinity, and out-of-range values.
- `SinhConvergenceError`: used if the series does not converge within the maximum iteration limit.

The GUI catches these exceptions and prints messages that explain what went wrong and how the user can correct the input.

1.7 Graphical user interface

The GUI is implemented using Tkinter. It includes:

- an input box for x,
- a Calculate button,
- a Clear button,
- an Exit button,
- a result area showing the input, the calculated value, algorithm name, and number of terms used,
- a status message for success or error conditions.

1.8 Manual validation examples

Input	Approximate output	Purpose
0	0	zero case
1	1.1752011936438	common positive input
-1	-1.1752011936438	common negative input
3	10.0178749274099	larger positive input
-3	-10.0178749274099	larger negative input
20	242582597.704895	upper boundary

1.9 Problem 5 CASTROFF prompt record

Prompt type: implementation-generation prompt with constraints and verification criteria.

```
Context: I am preparing SOEN 6011 Delivery 2 for F3, sinh(x). D1 used a textual Python
    implementation. D2 requires modifying it to implement the function from scratch in
    Python and adding a Tkinter GUI.
Audience: A course evaluator who will check that no Python math-library function is used
    for the function calculation.
```

```
Specific task: Generate a Python design and source code structure for sinh(x) using the
    Maclaurin series, with Tkinter input/output, custom exceptions, helpful error
    messages, and no math.sinh, math.exp, math.factorial, NumPy, or SciPy.
Tone: Clear, academic, and implementation-focused.
Requirements: Support finite real input, reject invalid input, explain the range, and
    separate parsing, calculation, and GUI responsibilities.
Output format: Provide code and a short explanation of the algorithm and exception
    handling.
Feedback loop: After producing the output, check whether each D2 constraint is satisfied.
Final check: Identify any built-in or library mathematical function that should be
    removed.
```

Output use: The output was used to structure the code, separate GUI and calculation responsibilities, and document why the Maclaurin series is appropriate for a scratch implementation. The final source code was manually reviewed and modified.

2 Problem 6: Public distributed version control system

The source code must be placed in a public distributed version control system. The intended hosting service is GitHub.

2.1 Repository address

Repository URL to include after upload:

```
https://github.com/<your-username>/soen6011-d2-f3-sinh
```

Before final submission, replace the placeholder URL with the actual public repository address.

2.2 Repository contents

The repository should contain:

- `D2_F3_sinh_gui.py`
- `README.md`
- `Arvind_Lakshmanan_40310757_D2_Report.pdf`
- `Arvind_Lakshmanan_40310757_D2_Slides.pdf`
- `commit_messages.md`

2.3 High-quality commit messages

High-quality commit messages should be specific, action-oriented, and connected to meaningful project changes. Suggested commit sequence:

1. `docs: add D2 README with setup and run instructions`
2. `feat: implement from-scratch Maclaurin series for sinh`
3. `feat: add Tkinter GUI with input validation and helpful errors`
4. `refactor: separate parsing, calculation, and GUI responsibilities`
5. `docs: update D2 requirements and traceability`

2.4 README content

The README file explains the project purpose, supported input range, run command, file list, example outputs, and the requirement to replace the placeholder GitHub URL with the actual public repository URL.

2.5 Problem 6 CASTROFF prompt record

Prompt type: documentation and version-control planning prompt.

```
Context: I need to prepare the D2 repository evidence for a SOEN 6011 project. The
    project is F3, sinh(x), implemented from scratch with a Tkinter GUI.
Audience: A professor or TA checking repository quality.
Specific task: Draft README content and high-quality Git commit messages. The README must
     explain how to run the program, supported input, files, and the from-scratch
    constraint.
Tone: Professional and concise.
Requirements: Include a placeholder for the public GitHub address and a reminder to
    replace it before submission.
Output format: Markdown README and a list of commit messages.
Feedback loop: Check whether the README includes purpose, run command, input range, and
    file list.
Final check: Ensure no claim says the repository already exists unless the public URL has
     been added.
```

Output use: The output was used to prepare `README.md` and `commit_messages.md`. The repository URL remains a placeholder until the project is uploaded to a public repository.

3 Problem 7: Updated D2 requirements

The D1 requirements are modified to account for the D2 scratch implementation and Tkinter GUI.

3.1 Definitions for D2

Term	D2 definition
From scratch	The function calculation must not use Python built-in or library mathematical functions such as `math.sinh`, `math.exp`, or `math.factorial`.
GUI	A graphical user interface created using Tkinter for entering input and viewing output.
Supported real input	One finite real number that can be represented as a Python floating-point value and lies in the interval $[-20, 20]$.
Helpful error message	A message that identifies the problem and guides the user toward a valid input.
Convergence	The Maclaurin series reaches the stopping condition before the maximum number of terms.

3.2 Updated requirements list

ID	Updated requirement	Rationale	Priority
REQ-F3-D2-01	The system shall provide a Tkinter graphical user interface for input and output.	D2 requires GUI support.	High
REQ-F3-D2-02	The system shall accept exactly one finite real number x through the GUI.	Prevents ambiguous input.	High
REQ-F3-D2-03	The system shall support inputs in the range $-20 \leq x \leq 20$.	Maintains reliable performance for the scratch series.	High
REQ-F3-D2-04	The system shall calculate $\sinh(x)$ using a from-scratch Maclaurin series implementation.	Satisfies the scratch requirement.	High
REQ-F3-D2-05	The system shall not use `math.sinh`, `math.exp`, `math.factorial`, NumPy, SciPy, or equivalent mathematical library functions for the calculation.	Prevents prohibited shortcuts.	High
REQ-F3-D2-06	The system shall stop the series when the term is smaller than the defined tolerance condition.	Supports controlled approximation.	High
REQ-F3-D2-07	The system shall raise and handle a helpful input error for blank, non-numeric, NaN, infinity, and out-of-range inputs.	Improves user experience.	High

ID	Updated requirement	Rationale	Priority
REQ-F3-D2-08	The system shall raise and handle a convergence error if the required tolerance is not reached within the maximum term count.	Handles computational failure safely.	Medium
REQ-F3-D2-09	The system shall display the input, calculated output, algorithm name, and number of terms used.	Gives transparent feedback to the user.	Medium
REQ-F3-D2-10	The system shall separate parsing, calculation, and GUI responsibilities in the source code.	Improves maintainability.	Medium
REQ-F3-D2-11	The source code shall run using standard Python without depending on a specific IDE.	Satisfies portability expectation.	High
REQ-F3-D2-12	The source code shall be placed in a public distributed version control system with a README file.	Satisfies D2 repository requirement.	High

3.3 Traceability from D1 to D2

D1 concern	D2 modification	Reason
Textual interface	Replaced by Tkinter GUI	D2 requires GUI.
Direct exponential formula	Replaced by Maclaurin series	D2 requires scratch implementation.
Use of `math.exp`	Removed from function calculation	Library math functions are not allowed for scratch calculation.
Input error handling	Expanded with custom exceptions	D2 requires exception handling and helpful messages.
Supported range $[-709, 709]$	Updated to $[-20, 20]$	Ensures reliable scratch-series performance.

3.4 Problem 7 CASTROFF prompt record

Prompt type: requirements-revision prompt.

```
Context: D1 requirements were written for a textual sinh(x) calculator using a direct
    exponential formula. D2 changes the implementation to a from-scratch Maclaurin series
     and a Tkinter GUI.
Audience: SOEN 6011 evaluator checking requirements traceability.
Specific task: Update the D1 requirements for D2. Requirements must cover GUI input/
```

```
    output, scratch calculation, input range, helpful exceptions, no math libraries,
    README, and public version control.
Tone: Formal requirements wording using shall statements.
Requirements: Provide unique identifiers, rationale, and priorities. Make implementation
    decisions explicit.
Output format: A requirements table and a traceability table from D1 to D2.
Feedback loop: Check whether every D2 Problem 5 and Problem 6 constraint is represented
    by at least one requirement.
Final check: Avoid unsupported claims and keep assumptions explicit.
```

Output use: The output was used to produce the updated requirements table and the D1-to-D2 traceability table. The wording was manually reviewed to keep each requirement verifiable.

4 Appendix: Python source code

```
"""
SOEN 6011 Delivery 2 - F3: Hyperbolic Sine sinh(x)
Student: Arvind Lakshmanan
Student ID: 40310757

This implementation calculates sinh(x) from scratch using the Maclaurin series:
    sinh(x) = x + x^3/3! + x^5/5! + ...

The mathematical implementation does not use math.sinh, math.exp, or any other
Python mathematical library function. Tkinter is used only for the graphical
user interface, as required for D2.
"""

import tkinter as tk
from tkinter import ttk

LOWER_LIMIT = -20.0
UPPER_LIMIT = 20.0
TOLERANCE = 0.000000000000001
MAX_TERMS = 200


class SinhInputError(Exception):
    """Raised when the user input cannot be used as a supported real number."""


class SinhConvergenceError(Exception):
    """Raised when the series does not reach the required tolerance."""
```

```
def absolute_value(value):
    """Return the non-negative size of a number without using abs()."""
    if value < 0:
        return -value
    return value


def is_nan(value):
    """Return True when value is NaN. NaN is the only float unequal to itself."""
    return value != value


def is_infinite_or_too_large(value):
    """Return True for positive or negative infinity and unsafe huge values."""
    return value > 1.0e308 or value < -1.0e308


def parse_supported_real_number(text):
    """Convert the user entry to a supported finite real number."""
    cleaned_text = text.strip()

    if cleaned_text == "":
        raise SinhInputError("Please enter one real number, such as -2, 0.5, or 3e-2.")

    try:
        x_value = float(cleaned_text)
    except ValueError as exc:
        raise SinhInputError(
            "The input must be one real number. Do not enter letters, commas, or multiple
                values."
        ) from exc

    if is_nan(x_value) or is_infinite_or_too_large(x_value):
        raise SinhInputError("The input must be a finite real number, not NaN or infinity.
            ")

    if x_value < LOWER_LIMIT or x_value > UPPER_LIMIT:
        raise SinhInputError(
            "This D2 from-scratch implementation supports only -20 <= x <= 20. "
            "Please enter a value inside this range."
        )

    return x_value
```

```
def calculate_sinh_from_scratch(x_value):
    """
    Calculate sinh(x) using the Maclaurin series.

    Recurrence used:
        term_0 = x
        term_n = term_(n-1) * x^2 / ((2n)(2n + 1))

    This avoids factorial and exponentiation library functions.
    """
    if x_value == 0.0:
        return 0.0, 1

    total = x_value
    term = x_value
    x_squared = x_value * x_value
    iteration = 1

    while iteration <= MAX_TERMS:
        denominator = (2 * iteration) * (2 * iteration + 1)
        term = term * x_squared / denominator
        total = total + term

        relative_stop = TOLERANCE * (1.0 + absolute_value(total))
        if absolute_value(term) <= relative_stop:
            return total, iteration + 1

        iteration = iteration + 1

    raise SinhConvergenceError(
        "The calculation did not converge within the iteration limit. "
        "Try an input closer to zero."
    )


class SinhCalculatorApp:
    """Tkinter graphical user interface for the F3 sinh calculator."""

    def __init__(self, root):
        self.root = root
        self.root.title("F3 Hyperbolic Sine Calculator - D2")
        self.root.geometry("640x420")
```

```
        self.root.minsize(560, 360)

        self.input_value = tk.StringVar()
        self.status_value = tk.StringVar(value="Enter a real number and click Calculate.")

        self.create_widgets()

    def create_widgets(self):
        main_frame = ttk.Frame(self.root, padding=20)
        main_frame.pack(fill="both", expand=True)

        title_label = ttk.Label(
            main_frame,
            text="F3: Hyperbolic Sine sinh(x)",
            font=("Arial", 18, "bold"),
        )
        title_label.pack(anchor="w")

        subtitle_label = ttk.Label(
            main_frame,
            text="From-scratch Maclaurin series implementation using Tkinter GUI",
        )
        subtitle_label.pack(anchor="w", pady=(2, 14))

        input_frame = ttk.Frame(main_frame)
        input_frame.pack(fill="x", pady=(0, 10))

        input_label = ttk.Label(input_frame, text="Enter x (-20 to 20):")
        input_label.pack(side="left")

        input_entry = ttk.Entry(input_frame, textvariable=self.input_value, width=28)
        input_entry.pack(side="left", padx=(10, 0))
        input_entry.focus()
        input_entry.bind("<Return>", self.calculate)

        button_frame = ttk.Frame(main_frame)
        button_frame.pack(fill="x", pady=(4, 12))

        calculate_button = ttk.Button(button_frame, text="Calculate", command=self.
            calculate)
        calculate_button.pack(side="left")

        clear_button = ttk.Button(button_frame, text="Clear", command=self.clear)
        clear_button.pack(side="left", padx=(8, 0))
```

```
        exit_button = ttk.Button(button_frame, text="Exit", command=self.root.destroy)
        exit_button.pack(side="left", padx=(8, 0))

        result_label = ttk.Label(main_frame, text="Result and explanation:")
        result_label.pack(anchor="w")

        self.result_box = tk.Text(main_frame, height=9, wrap="word")
        self.result_box.pack(fill="both", expand=True, pady=(5, 8))
        self.result_box.insert("1.0", "No calculation yet.")
        self.result_box.config(state="disabled")

        status_label = ttk.Label(main_frame, textvariable=self.status_value)
        status_label.pack(anchor="w")

    def set_result_text(self, message):
        self.result_box.config(state="normal")
        self.result_box.delete("1.0", "end")
        self.result_box.insert("1.0", message)
        self.result_box.config(state="disabled")

    def calculate(self, event=None):
        try:
            x_value = parse_supported_real_number(self.input_value.get())
            result, terms_used = calculate_sinh_from_scratch(x_value)
            message = (
                "Input x: " + format(x_value, ".15g") + "\n"
                "sinh(x): " + format(result, ".15g") + "\n"
                "Algorithm: Maclaurin series calculated from scratch\n"
                "Terms used: " + str(terms_used) + "\n\n"
                "Series used: sinh(x) = x + x^3/3! + x^5/5! + ..."
            )
            self.set_result_text(message)
            self.status_value.set("Calculation completed successfully.")
        except SinhInputError as error:
            self.set_result_text("Input error:\n" + str(error))
            self.status_value.set("Please correct the input and try again.")
        except SinhConvergenceError as error:
            self.set_result_text("Calculation error:\n" + str(error))
            self.status_value.set("The series did not reach the required tolerance.")
        except Exception as error:
            self.set_result_text("Unexpected error:\n" + str(error))
            self.status_value.set("An unexpected error occurred.")
```

```
    def clear(self):
        self.input_value.set("")
        self.set_result_text("No calculation yet.")
        self.status_value.set("Enter a real number and click Calculate.")


def main():
    root = tk.Tk()
    app = SinhCalculatorApp(root)
    root.mainloop()


if __name__ == "__main__":
    main()
```